

Chapter 10 Hash Tables

Hash Tables

◆ Prompt 1: What is a hash table?

✓ Answer:

A **hash table** is a data structure used to implement the **Map ADT** that allows:

- Fast insertion
- Fast deletion
- Fast lookup

☞ All in **$O(1)$ average time**

Python's built-in `dict` is implemented using a **hash table**.

◆ Prompt 2: What is the key idea behind a map?

✓ Answer:

A map lets us treat **keys like array indices**:

$M[k] = v$

Conceptually:

- Key $\rightarrow$ index
- Index $\rightarrow$ value

✦ Hash tables make this idea work even when:

- Keys are **not integers**
- Keys are **not small or consecutive**

◆ Prompt 3: What is a lookup table?

✓ Answer:

A **lookup table** is a simple array-based map used when:

- Keys are integers
- Keys fall in a small known range $[0, N-1]$

Example:

Keys: {1, 3, 6, 7}

Values: {D, Z, C, Q}

Index: 0 1 2 3 4 5 6 7 8 9 10

Value: - D - Z - - C Q - - -

✦ Access is immediate:

```
value = table[k]    # O(1)
```

◆ Prompt 4: What is a hash function?

✓ Answer:

A **hash function** converts a key into an integer index:

```
hash(key) → index in range [0, N-1]
```

Example:

```
index = hash(key) % N
```

✦ Goal:

- Distribute keys **uniformly**
- Minimize collisions

◆ Prompt 5: What is a collision?

✓ Answer:

A **collision** occurs when:

Two different keys map to the same index

```
hash(k1) == hash(k2)
```

Example:

```
hash(1) → 3
```

```
hash(25) → 3 ← collision
```

✦ Collisions are **unavoidable**, but manageable.

◆ Prompt 6: What is a bucket array?

✓ Answer:

A **bucket array** is an array where:

- Each index stores a **bucket**
- A bucket can hold **multiple key–value pairs**

Visual representation:

Index	Bucket
-------	--------

0	None
1	[(1,D) , (25,C)]
2	None
3	[(3,F) , (14,Z)]
4	None
5	None
6	[(6,A) , (39,C)]
7	[(7,Q)]
8	None
9	None
10	None

★ Each bucket handles collisions.

Hash Functions

◆ Prompt 1: What is the purpose of a hash function?

✓ Answer:

The purpose of a **hash function** h is to:

Map a key k to an integer index in the range $[0, N - 1]$, where N is the size of the hash table.

This index tells us **where to store or find** a key–value pair in the bucket array.

$(k, v) \rightarrow h(k) \rightarrow \text{index} \rightarrow \text{bucket}$

◆ Prompt 2: Why can't we use keys directly as array indices?

✓ Answer:

Because:

- Keys may **not be integers**
- Keys may be **very large**
- Keys may be **strings, tuples, or objects**

★ Hash functions solve this by converting arbitrary keys into valid indices.

◆ Prompt 3: How does a hash table use a hash function?

✓ Answer:

A hash table:

1. Computes $h(k)$
2. Uses that value as an index
3. Stores (k, v) in bucket $A[h(k)]$

$\text{index} = h(k)$

$A[\text{index}].\text{append}((k, v))$

◆ Prompt 4: What is a collision?

✓ Answer:

A **collision** occurs when:

Two different keys produce the same hash value

$h(k_1) == h(k_2)$, where $k_1 \neq k_2$

✦ Collisions are unavoidable because:

- Infinite keys
- Finite table size

◆ Prompt 5: What are the two parts of a hash function?

✓ Answer:

A hash function consists of **two stages**:

1. **Hash Code**
2. **Compression Function**

Key $\rightarrow$ Hash Code $\rightarrow$ Compression $\rightarrow$ Table Index

◆ Prompt 6: What is a hash code?

✓ Answer:

A **hash code** maps a key to an integer (possibly very large or negative).

Example:

`hash("cat")` $\rightarrow$ 823764298

`hash(42)` $\rightarrow$ 42

✦ Hash codes:

- Are **independent of table size**
- Can be reused for different hash tables

◆ Prompt 7: What is a compression function?

✓ Answer:

A **compression function** maps a hash code into the valid index range:

$[0, N - 1]$

Common method:

```
index = hash_code % N
```

✦ Compression ensures:

- Index fits inside the bucket array
- Table size can change dynamically

◆ Prompt 8: Why separate hash code and compression?

✓ Answer:

Separating them allows:

- ✓ Reusable hash codes
- ✓ Easy table resizing
- ✓ Better modular design

When the table grows:

- Only the compression function changes
- Hash codes remain valid

◆ Prompt 9: Example of hashing a key

✓ Answer:

Let:

- Key = "dog"
- `hash("dog") = 98451234`
- Table size $N = 11$

```
index = 98451234 % 11 = 6
```

✦ Store (dog, value) in A[6]

◆ Prompt 10: What happens when collisions occur?

✓ Answer:

If two keys map to the same index:

$A[6] = [(dog, 5), (cat, 9)]$

✦ Collision resolution strategies:

- Separate chaining
- Open addressing (linear probing, quadratic probing, etc.)

◆ Prompt 11: Time complexity impact of hash functions

✓ Answer:

Quality of Hash Function	Performance
Good distribution	$O(1)$ average
Poor distribution	$O(n)$ worst case

✦ Hash function quality is critical for efficiency.

Compression Functions (Hash Tables)

◆ Prompt 1: Why do we need a compression function?

✓ Answer:

A **hash code** alone cannot be used directly as an index because:

- It may be **negative**
- It may be **very large**
- It may exceed the table size

★ A **compression function** maps the hash code into the valid index range:

$[0, N - 1]$

where N is the bucket array size.

◆ Prompt 2: What is a compression function?

✓ Answer:

A **compression function** converts an integer hash code into a valid table index.

hash code $\rightarrow$ compression function $\rightarrow$ index

It is the **second step** of a hash function.

◆ Prompt 3: What makes a compression function “good”?

✓ Answer:

A good compression function:

- ✓ Minimizes collisions
- ✓ Spreads values uniformly
- ✓ Is fast to compute
- ✓ Works well with table resizing

★ Its goal is **uniform distribution** across buckets.

◆ Prompt 4: What is the Division Method?

✓ Answer:

The **division method** is the most common compression function.

Formula:

$$\text{index} = i \bmod N$$

where:

- i = hash code
- N = size of the bucket array

◆ Prompt 5: Why is the division method popular?

✓ Answer:

Because it is:

- ✓ Simple
- ✓ Fast
- ✓ Easy to implement
- ✓ Effective with a good choice of N

◆ Prompt 6: Why should N be a prime number?

✓ Answer:

Choosing N as a **prime number** helps:

- Reduce repeated patterns
- Spread hash values evenly
- Minimize collisions

✦ Non-prime N can cause clustering when hash codes follow patterns.

◆ Prompt 7: Example of poor choice of N

✓ Answer:

Hash codes:

$\{200, 205, 210, 215, 220, \dots, 600\}$

If $N = 100$:

$200 \bmod 100 = 0$

$205 \bmod 100 = 5$

$210 \bmod 100 = 10$

$215 \bmod 100 = 15$

$220 \bmod 100 = 20$

$\dots$

✦ These values repeat → collisions

◆ Prompt 8: Same example with prime N

✓ Answer:

If $N = 101$:

$200 \bmod 101 = 99$

$205 \bmod 101 = 3$

$210 \bmod 101 = 8$

$215 \bmod 101 = 13$

$220 \bmod 101 = 18$

✦ No repetition → better distribution

◆ Prompt 9: What is the ideal collision probability?

✓ Answer:

If the hash function is good:

$$P(\text{collision}) \approx 1 / N$$

★ Larger $N \rightarrow$ fewer collisions

★ Uniform distribution $\rightarrow$ predictable performance

Separate Chaining (Hash Tables)

◆ Prompt 1: What is Separate Chaining?

✓ Answer:

Separate chaining is a **collision resolution technique** used in hash tables.

When multiple keys hash to the same index:

- They are stored together in a **bucket**
- Each bucket is a **secondary data structure** (usually a list or map)

★ Instead of forcing all items into one slot, each slot stores a *collection* of items.

◆ Prompt 2: How is Separate Chaining implemented in ChainHashMap?

✓ Answer:

In ChainHashMap, each bucket is implemented using:

- UnsortedTableMap

```

1 class ChainHashMap(HashMapBase):
2     """Hash map implemented with separate chaining for collision resolution."""
3
4     def _bucket_getitem(self, j, k):
5         bucket = self._table[j]
6         if bucket is None:
7             raise KeyError('Key Error: ' + repr(k))        # no match found
8         return bucket[k]                                    # may raise KeyError
9
10    def _bucket_setitem(self, j, k, v):
11        if self._table[j] is None:
12            self._table[j] = UnsortedTableMap( )            # bucket is new to the table
13            oldsize = len(self._table[j])
14            self._table[j][k] = v
15            if len(self._table[j]) > oldsize:                # key was new to the table
16                self._n += 1                                # increase overall map size
17
18    def _bucket_delitem(self, j, k):
19        bucket = self._table[j]
20        if bucket is None:
21            raise KeyError('Key Error: ' + repr(k))        # no match found
22        del bucket[k]                                       # may raise KeyError
23
24    def __iter__(self):
25        for bucket in self._table:
26            if bucket is not None:                          # a nonempty slot
27                for key in bucket:
28                    yield key

```

Code Fragment 10.5: Concrete hash map class with separate chaining.

◆ Prompt 3: Why use `UnsortedTableMap` as a bucket?

✓ **Answer:**

Because:

- It already implements the **Map ADT**
- It supports `getitem`, `setitem`, `delitem`
- It is simple and flexible

★ Buckets are usually small, so linear-time operations are acceptable.

◆ Prompt 4: What is the role of `bucket_getitem`?

✓ Answer:

`bucket_getitem(j, k)` retrieves the value associated with key `k` in bucket `j`.

Code logic:

```
bucket = self._table[j]

if bucket is None:

    raise KeyError

return bucket[k]
```

Explanation:

- If bucket is empty → key not found
- Otherwise → delegate lookup to `UnsortedTableMap`

◆ Prompt 5: What is the role of `bucket_setitem`?

✓ Answer:

`bucket_setitem(j, k, v)` inserts or updates a key–value pair in bucket `j`.

Code logic:

```
if self._table[j] is None:

    self._table[j] = UnsortedTableMap()

oldsize = len(self._table[j])

self._table[j][k] = v

if len(self._table[j]) > oldsize:

    self._n += 1
```

Key idea:

- Only increase map size (`self._n`) if the key is new

◆ Prompt 6: Why do we compare bucket size before and after insertion?

✓ Answer:

To determine whether:

- A **new key** was added
- Or an **existing key** was updated

✦ The total number of keys in the hash map must stay correct.

◆ Prompt 7: What is the role of `bucket_delitem`?

✓ Answer:

`bucket_delitem(j, k)` removes key `k` from bucket `j`.

Code logic:

```
bucket = self._table[j]

if bucket is None:
    raise KeyError

del bucket[k]
```

✦ Deletion is delegated to the bucket's map.

◆ Prompt 8: How does iteration work in `ChainHashMap`?

✓ Answer:

The `__iter__` method:

- Scans every bucket
- Iterates over keys in non-empty buckets

Code logic:

```
for bucket in self._table:
```

```
if bucket is not None:
    for key in bucket:
        yield key
```

✦ This returns **all keys** in the hash map.

Linear Probing

◆ Prompt 1: What is Linear Probing?

✓ Answer:

Linear probing is a **collision resolution technique** in hash tables that uses **open addressing**.

✦ When a collision occurs:

- We move to the **next slot**
- Continue sequentially until an empty slot is found

Formula:

$(h(k) + i) \bmod N$ where $i = 0, 1, 2, \dots$

◆ Prompt 2: What is Open Addressing?

✓ Answer:

In **open addressing**:

- All key-value pairs are stored **inside the main table**
- No secondary data structures (no buckets)

✦ One table, one array — collisions are resolved by probing.

```

1 class ProbeHashMap(HashMapBase):
2     """ Hash map implemented with linear probing for collision resolution."""
3     _AVAIL = object( )      # sentinel marks locations of previous deletions
4
5     def _is_available(self, j):
6         """ Return True if index j is available in table."""
7         return self._table[j] is None or self._table[j] is ProbeHashMap._AVAIL
8
9     def _find_slot(self, j, k):
10        """ Search for key k in bucket at index j.
11
12        Return (success, index) tuple, described as follows:
13        If match was found, success is True and index denotes its location.
14        If no match found, success is False and index denotes first available slot.
15        """
16        firstAvail = None
17        while True:
18            if self._is_available(j):
19                if firstAvail is None:
20                    firstAvail = j                # mark this as first avail
21                if self._table[j] is None:
22                    return (False, firstAvail)    # search has failed
23                elif k == self._table[j]._key:
24                    return (True, j)             # found a match
25                j = (j + 1) % len(self._table)   # keep looking (cyclically)

```

Code Fragment 10.6: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued in Code Fragment 10.7).


```

26 def _bucket_getitem(self, j, k):
27     found, s = self._find_slot(j, k)
28     if not found:
29         raise KeyError('Key Error: ' + repr(k))           # no match found
30     return self._table[s]._value
31
32 def _bucket_setitem(self, j, k, v):
33     found, s = self._find_slot(j, k)
34     if not found:
35         self._table[s] = self._Item(k,v)                   # insert new item
36         self._n += 1                                       # size has increased
37     else:
38         self._table[s]._value = v                          # overwrite existing
39
40 def _bucket_delitem(self, j, k):
41     found, s = self._find_slot(j, k)
42     if not found:
43         raise KeyError('Key Error: ' + repr(k))           # no match found
44     self._table[s] = ProbeHashMap._AVAIL                  # mark as vacated
45
46 def __iter__(self):
47     for j in range(len(self._table)):                       # scan entire table
48         if not self._is_available(j):
49             yield self._table[j]._key

```

Code Fragment 10.7: Concrete ProbeHashMap class that uses linear probing for collision resolution (continued from Code Fragment 10.6).

◆ Prompt 3: Why do we need a special `AVAIL` marker?

✓ Answer:

When deleting elements:

- We cannot set the slot to `None`
- Because it would break future searches

✦ Solution: Use a **sentinel marker** (`AVAIL`)

```
AVAIL = object()
```

This distinguishes:

Slot	Meaning
------	---------

State	
None	Never used
AVAIL	Previously used, now deleted
(k, v)	Active item

◆ Prompt 4: What does `is_available(j)` check?

✓ Answer:

It checks whether index `j` can be used for insertion.

```
return self._table[j] is None or self._table[j] is AVAIL
```

✦ Available slots:

- Empty (`None`)
- Deleted (`AVAIL`)

◆ Prompt 5: What is the purpose of `findslot(j, k)`?

✓ Answer:

`findslot`:

- Searches for key `k`
- Finds where to insert or retrieve it

It returns:

```
(success, index)
```

Case	Meaning
<code>success = True</code>	Key found
<code>success = False</code>	Key not found
<code>index</code>	Slot location

◆ Prompt 6: How does `findslot` work step by step?

✓ Answer:

1. Start probing at index `j`
2. Track the first `AVAIL` slot
3. Continue probing **until None is found**
4. Wrap around using modulo

```
j = (j + 1) % len(self._table)
```

✦ This creates **cyclic probing**

◆ Prompt 7: Why can't search stop at `AVAIL`?

✓ Answer:

Because:

- The key **may have been inserted after that slot**
- Deleting creates gaps

✦ Search only stops when `None` is found.

◆ Prompt 8: How does `bucket_getitem` work?

✓ Answer:

```
found, s = self.findslot(j, k)
```

```
if not found:
```

```
    raise KeyError
```

```
return self._table[s].value
```

✦ Uses `findslot` to:

- Locate key
- Retrieve value

◆ Prompt 9: How does `bucket_setitem` work?

✓ Answer:

```
found, s = self.findslot(j, k)

if not found:

    self._table[s] = Item(k, v)

    self._n += 1

else:

    self._table[s].value = v
```

✦ Logic:

- Overwrite if key exists
- Insert at first available slot if new

◆ Prompt 10: How does deletion work in Linear Probing?

✓ Answer:

```
self._table[s] = AVAIL
```

✦ We do NOT remove the slot:

- Mark it as **vacated**
- Preserve probe sequences

◆ Prompt 11: How does iteration work?

✓ Answer:

```
for j in range(len(self._table)):

    if not self.is_available(j):

        yield self._table[j].key
```

✦ Skips:

- None
- AVAIL